After validating the arguments received (omitted in listing 4.9), we can start setting a few base fields; then it comes the trickiest part: given the number of elements and the expected precision, compute how large our buffer needs to be: we use the formula described in the section 4.10, but we need to verify that the size of the buffer is still safe to be held in memory.

Once we have the size of the buffer, we can compute the optimal number of hashes needed to keep the rate of false positives as low as possible.

4.6.6 Checking a Key

We can now start composing the helper methods presented so far, to build the Bloom filter's API methods. One note of caution: we assume keys are going to be strings, but they can also be serializable objects; if that's the case, you will need a consistent serialization function, that turns equivalent objects (for instance, two sets containing the same elements) into the same string, otherwise no matter how good it is your implementation of the Bloom filter (or any other dictionary you might use), your application won't work properly.

Data massaging and preprocessing are often as important, or more, than the actual algorithms run on it.
